第5次实验课作业

UTXO交易模型的 P2PKH 执行模式模拟

姓名	黄华威
学号	待填写
实验内容	A 向 B 转账 5 BTC，并验证 P2PKH 交易
完成日期	2026年10月12日

说明：资料中未提供学号，封面及文件夹名称中的“学号待填写”需要提交前替换为本人学号。

一、实验目标与交易流程

本实验使用 Python 模拟一笔完整的 UTXO 交易：A 拥有一个 10 BTC 的 P2PKH 锁定输出，A 向 B 支付 5 BTC，并将 4.99 BTC 作为找零返回 A，剩余 0.01 BTC 作为手续费。交易进入全局 UTXO 集前，由其他共识节点 someone 检查输入 UTXO、P2PKH 公钥哈希、ECDSA 签名以及输入输出金额关系。

核心步骤

Step I	构造未签名输入，引用 genesis_txid:0。
Step II	构造支付给 B 的 5.0 BTC 输出，并用 B 的公钥哈希锁定。
Step III	构造支付给 A 的 4.99 BTC 找零输出。
Step IV	组合输入和两个输出，创建 raw_tx。
Step V	计算 sighash，使用 A 私钥签名，将签名和公钥写入 script_sig，再计算 txid。

验证逻辑

someone 调用 raw_tx.verify(utxo_set)。验证成功后才允许删除被花费的旧 UTXO，并将两个新输出写回全局 UTXO 集；若验证失败，则抛出异常并停止更新。

本报告中的代码高亮表示本次补全的关键代码；完整可执行 Python 文件与本 PDF 位于同一文件夹。

二、补全代码高亮展示

以下为主程序中需要补全的 Step I-V。黄色背景为本次补全内容。

```
    # ===== 3.          tx:< A → B, 5> =====
    print("\n         tx: A → B (5 BTC)")

    # Step I:          txin    A   UTXO   Output
    #      “   ”    script_sig          sighash
    txin = TxIn(
        txid=genesis_txid,
        vout=0,
        script_sig={}
    )


    # Step II:            out1     B 5   BTC
    #   BTC “  ”     “    ” pubkey_hash
    #       P2PKH          script_pubkey       OP_DUP OP_HASH160 <pubkey_hash> OP_EQUALVERIFY OP_CHECKSIG
    out1 = TxOut(
        value=5.0,
        script_pubkey={
            "type": ScriptType.P2PKH,
            "hash": hash160(bytes.fromhex(B_public_key)).hex()
        }
    )


    # Step III:            out2     A (10 - 5 - 0.01 fee = 4.99 BTC)
    out2 = TxOut(
        value=4.99,
        script_pubkey={
            "type": ScriptType.P2PKH,
            "hash": hash160(bytes.fromhex(A_public_key)).hex()
        }
    )


    # Step IV:    “    ” raw_tx
```

三、交易验证与 UTXO 集更新代码

以下代码完成了“其他共识节点 someone 验证交易”的要求，并保证只有验证通过后才更新 UTXO 集。

```
    #        sign(sighash, A_private_key)          <signature> <pubkey>    txin.script_sig
    #            txid
    prev_script_code = utxo_set[f"{txin.txid}:{txin.vout}"].script_pubkey
    sighash = raw_tx.get_sighash(input_index=0, script_code=prev_script_code)
    signature = sign(sighash, A_private_key)

    #               script_sig <signature> <pubkey>
    txin.script_sig = {
        "signature": signature,
        "pubkey": A_public_key
    }

    #   “    ” raw_tx            txid
    raw_tx.txid = raw_tx.get_hash() ##             #      txid

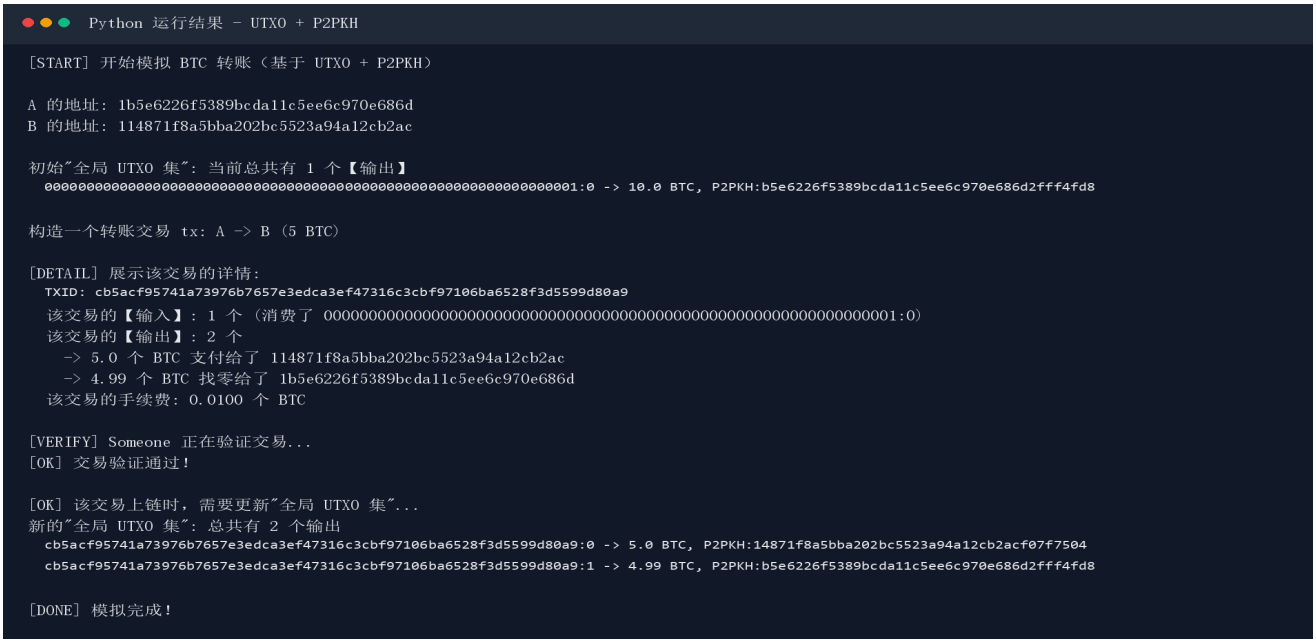
    print(f"\n          :")
    print(f"  TXID: {raw_tx.txid}")
    print(f"          : 1   (    {txin.txid}:{txin.vout})")
```

关键检查点

1	输入引用的 UTXO 必须存在且未被花费。
2	script_sig 中公钥的 HASH160 必须与旧输出的锁定哈希一致。
3	ECDSA 签名必须能用对应公钥验证通过。
4	输出总额不能超过输入总额。

四、代码运行截图

使用 Python 3 执行补全后的代码，交易随机生成 A、B 密钥对，因此每次运行的地址和 TXID 会不同；交易金额、手续费、验证结果和最终 UTXO 数量应保持一致。



运行结果：交易验证通过；10 BTC 输入被消费，生成 5.0 BTC 的 B 输出和 4.99 BTC 的 A 找零输出，最终全局 UTXO 集包含 2 个输出。

五、实验收获

通过本次练习，我理解了 UTXO 不是账户余额，而是可以被交易输入引用的一组未花费交易输出。一次转账需要消费完整的旧输出，并通过两个新输出完成收款和找零，手续费则体现为输入总额与输出总额之间的差额。

我还理解了 P2PKH 验证的基本顺序：先用公钥计算 HASH160 并与锁定脚本中的哈希比较，再对交易 sighash 验证签名。签名不能直接对最终带有 scriptSig 的交易随意计算，而要先用被花费输出的 scriptPubKey 构造待签名摘要，这也是代码中 prev_script_code 和 get_sighash 的作用。

最后，验证交易和更新 UTXO 集必须分成两个阶段：只有 someone 验证通过，才能删除旧 UTXO 并写入新 UTXO。这样可以避免无效交易污染全局状态。

提交前检查

请将文件夹名称和其中两个文件名的“学号待填写”替换为真实学号；提交时保留 Python 文件与本 PDF 在同一个文件夹内。